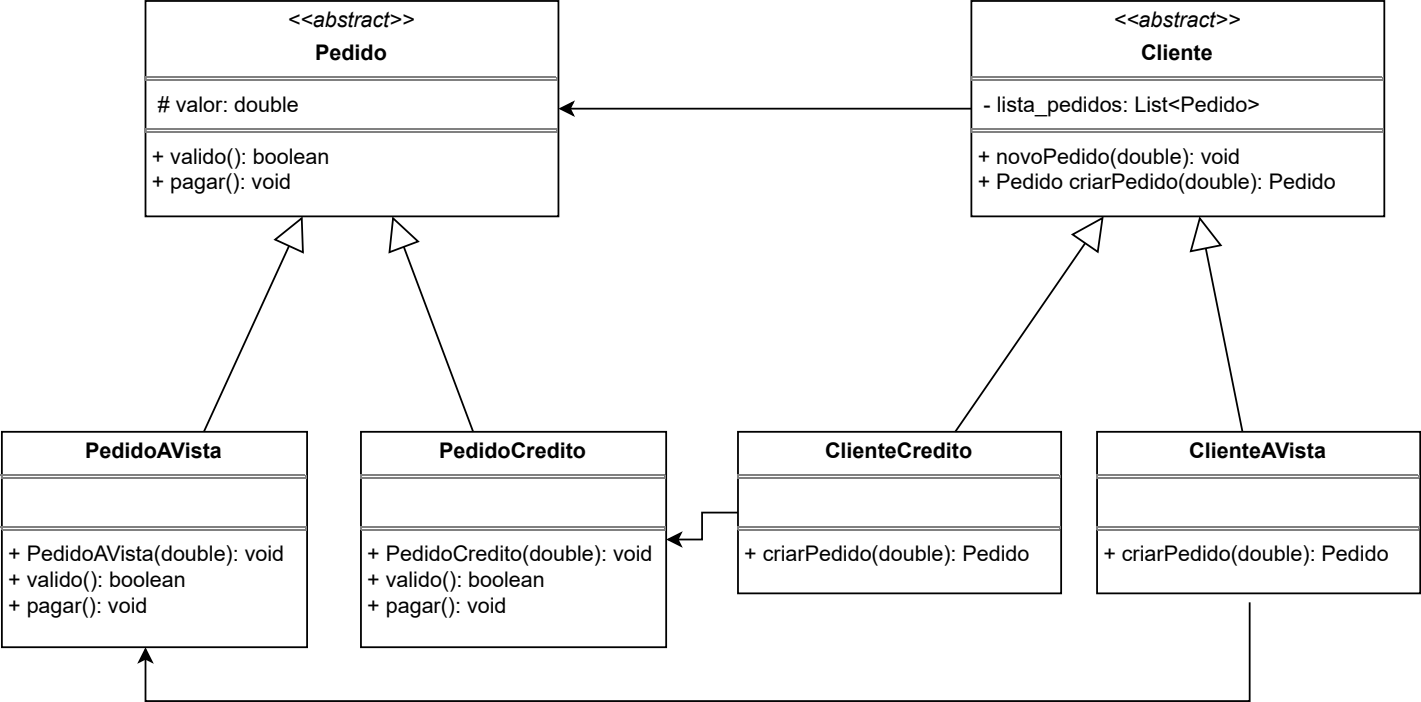




1. Qual classe representa o criador abstrato?

R.: Cliente.

2. Quais classes representam os criadores concretos?

R.: ClienteAVista e ClienteCredito.

3. Qual classe representa o produto abstrato?

R.: Pedido.

4. Quais classes representam os produtos concretos?

R.: PedidoAVista e PedidoCredito.

5. Qual método representa o Factory Method?

R.: criarPedido().

6. Por que criarPedido() deve ser abstrato?

R.: Porque cada tipo de cliente cria um tipo diferente de pedido.

7. Por que novoPedido() deve ser concreto?

R.: Pois é comum para todos os clientes.

8. Qual classe decide criar um PedidoAVista?

R.: ClienteAVista.

9. Qual classe decide criar um PedidoCredito?

R.: ClienteCredito.

10. Por que o programa principal pode trabalhar apenas com o tipo abstrato Cliente?

R.: Pois ele delega a criação dos objetos para as subclasses do Cliente.

11. Qual é a vantagem de Cliente manipular objetos do tipo Pedido em vez de classes concretas?

R.: Permite adicionar novos tipos de pedidos sem alterar a classe Cliente.

12. Por que este projeto é um exemplo do padrão Factory Method?

R.: Porque a criação dos objetos Pedido é delegada às subclasses de Cliente.

13. Qual é a diferença principal entre Factory Method e Simple Factory?

R.: No Factory Method a criação é delegada às subclasses, já no Simple Factory uma única classe centraliza toda a criação dos objetos.

14. O que aconteceria se a criação dos pedidos fosse feita diretamente no programa principal?

R.: O programa ficaria fortemente acoplado às classes concretas.

15. Como o sistema poderia ser estendido para incluir um novo tipo de cliente, por exemplo

ClienteFinanciamento?

R.: Uma classe ClienteFinanciamento que herda de Cliente. Uma classe PedidoFinanciamento que herda de Pedido. Implementando criarPedido() em ClienteFinanciamento.